

Support Vector Machines

Maximal Margins, Soft Margins, and the
Kernel Trick

Dr. Rajesh Kumar Mundotiya

Assistant Professor, Department of Computer Science
and Engineering

Lab In-Charge: MATRA Lab, Indian Institute of
Technology Bhilai



MATRA LAB

Session Roadmap

Recap & Objectives

Mathematical Prerequisites

The Maximal Margin Classifier

The Support Vector Classifier

Support Vector Machines: The Kernel Trick

Applied View

Summary



SECTION 1

Recap & Objectives

Where We Were: Tree Ensembles

From the previous session

Bagging and **random forests** average many high-variance trees grown on resampled data to reduce *variance*; **boosting** grows trees sequentially, each fixing the previous ensemble's residual error, to reduce *bias*. Both build a complex model out of many simple ones.

The gap this session fills

We now take a completely different, **geometric** route to classification: instead of averaging trees, we search directly for the separating boundary with the largest **margin** between classes. This single idea — margin maximization — extends naturally, via the **kernel trick**, to highly non-linear decision boundaries without ever growing a tree.

Where We Were: Tree Ensembles

From the previous session

Bagging and **random forests** average many high-variance trees grown on resampled data to reduce *variance*; **boosting** grows trees sequentially, each fixing the previous ensemble's residual error, to reduce *bias*. Both build a complex model out of many simple ones.

The gap this session fills

We now take a completely different, **geometric** route to classification: instead of averaging trees, we search directly for the separating boundary with the largest **margin** between classes. This single idea — margin maximization — extends naturally, via the **kernel trick**, to highly non-linear decision boundaries without ever growing a tree.

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Learning Objectives

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Learning Objectives

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Learning Objectives

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Learning Objectives

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Learning Objectives

By the end of this session you will be able to:

1. Define a hyperplane classifier and derive the distance from a point to a hyperplane.
2. Derive the maximal margin classifier and identify support vectors.
3. Explain why the maximal margin classifier fails on non-separable data, and how slack variables and the cost parameter C fix this.
4. Explain the kernel trick and compute with polynomial and radial kernels.
5. Extend binary SVMs to multiclass problems (OvO / OvR).

Motivation: Why a New Classifier?

Limitations of what we already know

- ▶ **Logistic regression**: linear boundary, can struggle when classes are well but not *linearly* separated.
- ▶ **Decision trees**: capture complex patterns but suffer from high variance and overfitting.
- ▶ **k-NN**: simple and non-parametric, but performance degrades in high dimensions.

Key idea

Support Vector Machines classify data by finding the **optimal separating hyperplane** that maximizes the margin between classes — a classifier that generalizes well, is robust to noise, and works effectively in high dimensions.

Motivation: Why a New Classifier?

Limitations of what we already know

- ▶ **Logistic regression**: linear boundary, can struggle when classes are well but not *linearly* separated.
- ▶ **Decision trees**: capture complex patterns but suffer from high variance and overfitting.
- ▶ **k-NN**: simple and non-parametric, but performance degrades in high dimensions.

Key idea

Support Vector Machines classify data by finding the **optimal separating hyperplane** that maximizes the margin between classes — a classifier that generalizes well, is robust to noise, and works effectively in high dimensions.

The Mathematics We Need First

1. Hyperplanes

For a p -dimensional feature space, a **hyperplane** is the flat $(p - 1)$ -dimensional set

$$\mathbf{x}^T \boldsymbol{\beta} + \beta_0 = 0.$$

It splits $\mathbb{R}^p$ into two half-spaces: $\mathbf{x}^T \boldsymbol{\beta} + \beta_0 > 0$ on one side, < 0 on the other.

2. Distance from a point to a hyperplane

The signed perpendicular distance from $\mathbf{x}$ to the hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$ is $\frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|}$.

Scaling $(\mathbf{w}, b)$ by a constant leaves the hyperplane unchanged but rescales this distance — a normalization we exploit shortly.

The Mathematics We Need First

1. Hyperplanes

For a p -dimensional feature space, a **hyperplane** is the flat $(p - 1)$ -dimensional set

$$\mathbf{x}^T \boldsymbol{\beta} + \beta_0 = 0.$$

It splits $\mathbb{R}^p$ into two half-spaces: $\mathbf{x}^T \boldsymbol{\beta} + \beta_0 > 0$ on one side, < 0 on the other.

2. Distance from a point to a hyperplane

The signed perpendicular distance from $\mathbf{x}$ to the hyperplane $\mathbf{w}^T \mathbf{x} + b = 0$ is $\frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|}$.

Scaling $(\mathbf{w}, b)$ by a constant leaves the hyperplane unchanged but rescales this distance — a normalization we exploit shortly.

3. Constrained optimization (intuition)

To *maximize* a quantity subject to inequality constraints, we look for the point where the objective's gradient is balanced against the *active* constraints — the ones holding with equality at the optimum. Here, those active constraints will be exactly the training points closest to the boundary.

4. Convex quadratic objectives

Minimizing $\frac{1}{2} \|\mathbf{w}\|^2$ subject to linear inequality constraints is a **convex quadratic program**: it has a unique global minimum, solvable reliably and efficiently.

3. Constrained optimization (intuition)

To *maximize* a quantity subject to inequality constraints, we look for the point where the objective's gradient is balanced against the *active* constraints — the ones holding with equality at the optimum. Here, those active constraints will be exactly the training points closest to the boundary.

4. Convex quadratic objectives

Minimizing $\frac{1}{2} \|\mathbf{w}\|^2$ subject to linear inequality constraints is a **convex quadratic program**: it has a unique global minimum, solvable reliably and efficiently.

What Is a Hyperplane? (Classification View)

Setup

For a dataset with p features, the decision boundary is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Classification rule

$$\mathbf{w}^T \mathbf{x} + b > 0 \Rightarrow \text{class } (+1), \quad \mathbf{w}^T \mathbf{x} + b < 0 \Rightarrow \text{class } (-1).$$

- ▶ $\mathbf{x}$: feature vector. $\mathbf{w}$: normal vector (orientation). b : bias / intercept (offset from origin).

What Is a Hyperplane? (Classification View)

Setup

For a dataset with p features, the decision boundary is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Classification rule

$$\mathbf{w}^T \mathbf{x} + b > 0 \Rightarrow \text{class } (+1), \quad \mathbf{w}^T \mathbf{x} + b < 0 \Rightarrow \text{class } (-1).$$

- ▶ $\mathbf{x}$: feature vector. $\mathbf{w}$: normal vector (orientation). b : bias / intercept (offset from origin).

What Is a Hyperplane? (Classification View)

Setup

For a dataset with p features, the decision boundary is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

Classification rule

$$\mathbf{w}^T \mathbf{x} + b > 0 \Rightarrow \text{class } (+1), \quad \mathbf{w}^T \mathbf{x} + b < 0 \Rightarrow \text{class } (-1).$$

- ▶ $\mathbf{x}$: feature vector. $\mathbf{w}$: normal vector (orientation). b : bias / intercept (offset from origin).

Which Hyperplane Should We Choose?

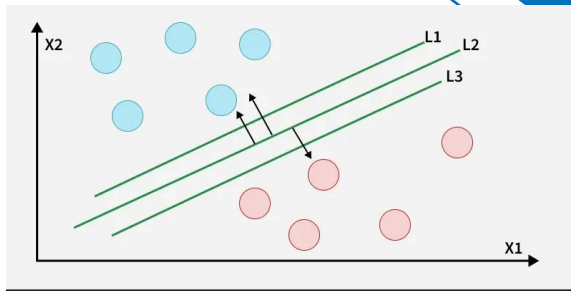
Observation

If the data are **linearly separable**, infinitely many hyperplanes achieve zero training error.

- ▶ They do *not* generalize equally well.
- ▶ Some are highly sensitive to small changes in the data.

Key question

Among all separating hyperplanes, which is expected to perform best on unseen data?



Several hyperplanes separate the data perfectly; only one gives the maximum margin.

Which Hyperplane Should We Choose?

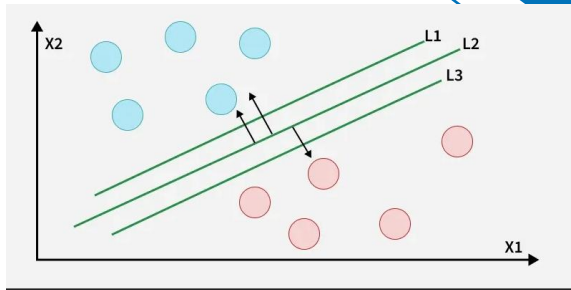
Observation

If the data are **linearly separable**, infinitely many hyperplanes achieve zero training error.

- ▶ They do *not* generalize equally well.
- ▶ Some are highly sensitive to small changes in the data.

Key question

Among all separating hyperplanes, which is expected to perform best on unseen data?



Several hyperplanes separate the data perfectly; only one gives the maximum margin.

Which Hyperplane Should We Choose?

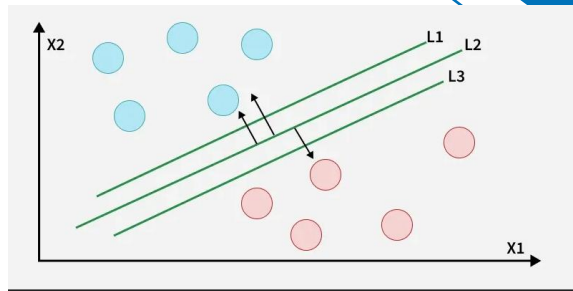
Observation

If the data are **linearly separable**, infinitely many hyperplanes achieve zero training error.

- ▶ They do *not* generalize equally well.
- ▶ Some are highly sensitive to small changes in the data.

Key question

Among all separating hyperplanes, which is expected to perform best on unseen data?



Several hyperplanes separate the data perfectly; only one gives the maximum margin.

Maximum Margin Classifier

Key idea

Among all separating hyperplanes, the **maximal margin classifier** picks the one that leaves the **largest possible margin** between the two classes.

Margin

The **margin** is the minimum perpendicular distance between the decision boundary and the nearest training observations.

- ▶ Improves generalization to unseen data.
- ▶ Makes the boundary less sensitive to small perturbations.
- ▶ Reduces overfitting risk; produces a more robust classifier.

Larger Margin $\implies$ Better Generalization $\implies$ Lower Test Error

Maximum Margin Classifier

Key idea

Among all separating hyperplanes, the **maximal margin classifier** picks the one that leaves the **largest possible margin** between the two classes.

Margin

The **margin** is the minimum perpendicular distance between the decision boundary and the nearest training observations.

- ▶ Improves generalization to unseen data.
- ▶ Makes the boundary less sensitive to small perturbations.
- ▶ Reduces overfitting risk; produces a more robust classifier.

Larger Margin $\implies$ Better Generalization $\implies$ Lower Test Error

Maximum Margin Classifier

Key idea

Among all separating hyperplanes, the **maximal margin classifier** picks the one that leaves the **largest possible margin** between the two classes.

Margin

The **margin** is the minimum perpendicular distance between the decision boundary and the nearest training observations.

- ▶ Improves generalization to unseen data.
- ▶ Makes the boundary less sensitive to small perturbations.
- ▶ Reduces overfitting risk; produces a more robust classifier.

Larger Margin $\implies$ Better Generalization $\implies$ Lower Test Error

Maximum Margin Classifier

Key idea

Among all separating hyperplanes, the **maximal margin classifier** picks the one that leaves the **largest possible margin** between the two classes.

Margin

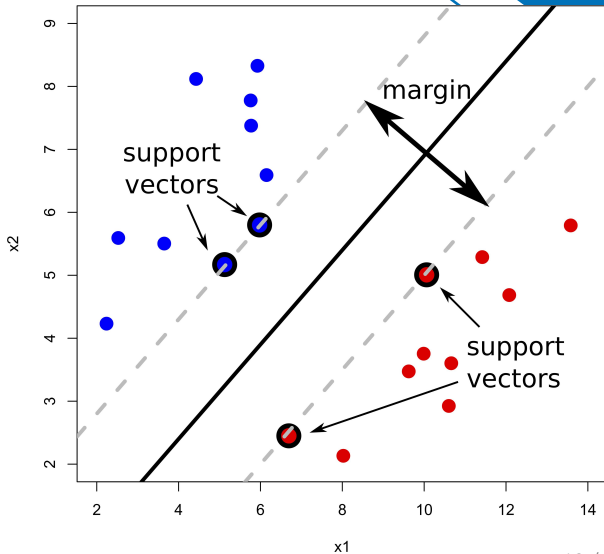
The **margin** is the minimum perpendicular distance between the decision boundary and the nearest training observations.

- ▶ Improves generalization to unseen data.
- ▶ Makes the boundary less sensitive to small perturbations.
- ▶ Reduces overfitting risk; produces a more robust classifier.

Larger Margin $\implies$ Better Generalization $\implies$ Lower Test Error

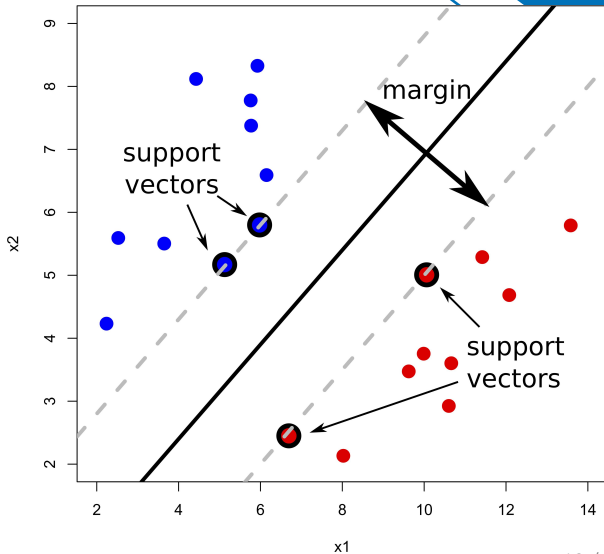
Geometry of the Maximum Margin Classifier

- ▶ The central line is the **decision boundary** (hyperplane).
- ▶ Two parallel lines define the **margin boundaries**.
- ▶ Points lying exactly on the margin are the **support vectors**.
- ▶ The margin is the perpendicular distance from the boundary to the nearest training samples.
- ▶ Only the support vectors determine the optimal hyperplane.



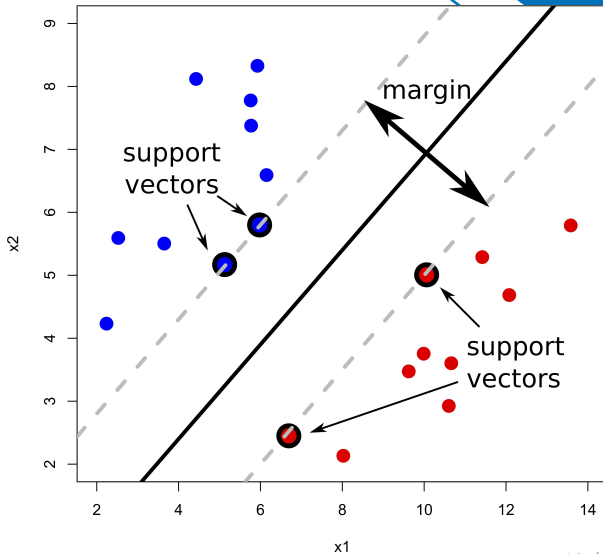
Geometry of the Maximum Margin Classifier

- ▶ The central line is the **decision boundary** (hyperplane).
- ▶ Two parallel lines define the **margin boundaries**.
- ▶ Points lying exactly on the margin are the **support vectors**.
- ▶ The margin is the perpendicular distance from the boundary to the nearest training samples.
- ▶ Only the support vectors determine the optimal hyperplane.



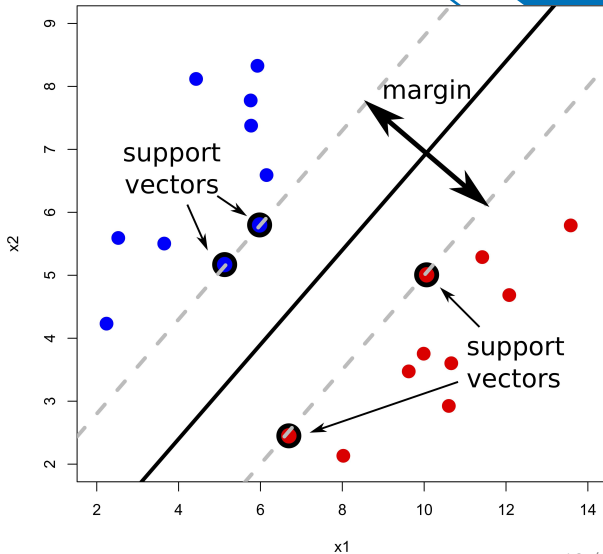
Geometry of the Maximum Margin Classifier

- ▶ The central line is the **decision boundary** (hyperplane).
- ▶ Two parallel lines define the **margin boundaries**.
- ▶ Points lying exactly on the margin are the **support vectors**.
- ▶ The margin is the perpendicular distance from the boundary to the nearest training samples.
- ▶ Only the support vectors determine the optimal hyperplane.



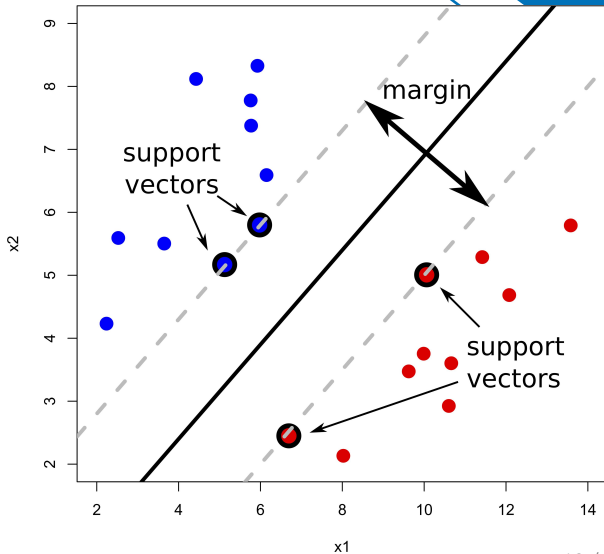
Geometry of the Maximum Margin Classifier

- ▶ The central line is the **decision boundary** (hyperplane).
- ▶ Two parallel lines define the **margin boundaries**.
- ▶ Points lying exactly on the margin are the **support vectors**.
- ▶ The margin is the perpendicular distance from the boundary to the nearest training samples.
- ▶ Only the support vectors determine the optimal hyperplane.



Geometry of the Maximum Margin Classifier

- ▶ The central line is the **decision boundary** (hyperplane).
- ▶ Two parallel lines define the **margin boundaries**.
- ▶ Points lying exactly on the margin are the **support vectors**.
- ▶ The margin is the perpendicular distance from the boundary to the nearest training samples.
- ▶ Only the support vectors determine the optimal hyperplane.



Mathematical Definition of the Margin

The decision boundary is $\mathbf{w}^T \mathbf{x} + b = 0$; the two margin boundaries are

$$\mathbf{w}^T \mathbf{x} + b = 1 \quad \text{and} \quad \mathbf{w}^T \mathbf{x} + b = -1.$$

By prerequisite 2, the distance from a point to the boundary is $\frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$, so the total distance between the two margin boundaries is

$$\text{Total Margin} = \frac{2}{\|\mathbf{w}\|}.$$

Mathematical Definition of the Margin

The decision boundary is $\mathbf{w}^T \mathbf{x} + b = 0$; the two margin boundaries are

$$\mathbf{w}^T \mathbf{x} + b = 1 \quad \text{and} \quad \mathbf{w}^T \mathbf{x} + b = -1.$$

By prerequisite 2, the distance from a point to the boundary is $\frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$, so the total distance between the two margin boundaries is

$$\text{Total Margin} = \frac{2}{\|\mathbf{w}\|}.$$

Optimization Objective

Maximizing the margin

$$\max \frac{2}{\|\mathbf{w}\|} \iff \min \|\mathbf{w}\| \iff \boxed{\min \frac{1}{2} \|\mathbf{w}\|^2}$$

(the square is for mathematical convenience — a smooth convex objective, per prerequisite 4).

Subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

- ▶ $\mathbf{x}_i$: feature vector of sample i . $y_i \in \{-1, +1\}$: class label.
- ▶ The constraint ensures every point is correctly classified and lies on or outside the margin.

Optimization Objective

Maximizing the margin

$$\max \frac{2}{\|\mathbf{w}\|} \iff \min \|\mathbf{w}\| \iff \boxed{\min \frac{1}{2} \|\mathbf{w}\|^2}$$

(the square is for mathematical convenience — a smooth convex objective, per prerequisite 4).

Subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

- ▶ $\mathbf{x}_i$: feature vector of sample i . $y_i \in \{-1, +1\}$: class label.
- ▶ The constraint ensures every point is correctly classified and lies on or outside the margin.

Optimization Objective

Maximizing the margin

$$\max \frac{2}{\|\mathbf{w}\|} \iff \min \|\mathbf{w}\| \iff \boxed{\min \frac{1}{2} \|\mathbf{w}\|^2}$$

(the square is for mathematical convenience — a smooth convex objective, per prerequisite 4).

Subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n.$$

- ▶ $\mathbf{x}_i$: feature vector of sample i . $y_i \in \{-1, +1\}$: class label.
- ▶ The constraint ensures every point is correctly classified and lies on or outside the margin.

Why the Constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$?

The margin boundaries are $\mathbf{w}^\top \mathbf{x} + b = 1$ and $\mathbf{w}^\top \mathbf{x} + b = -1$. Therefore

$$y_i = +1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \geq 1, \quad y_i = -1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \leq -1.$$

Combining both cases into one expression:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Support vectors

Points where this constraint holds with *equality*, $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, are the **support vectors**: the closest points to the boundary, and the *only* points that determine it. Removing a non-support vector leaves the classifier unchanged; moving a support vector moves the boundary.

Why the Constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$?

The margin boundaries are $\mathbf{w}^\top \mathbf{x} + b = 1$ and $\mathbf{w}^\top \mathbf{x} + b = -1$. Therefore

$$y_i = +1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \geq 1, \quad y_i = -1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \leq -1.$$

Combining both cases into one expression:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Support vectors

Points where this constraint holds with *equality*, $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, are the **support vectors**: the closest points to the boundary, and the *only* points that determine it. Removing a non-support vector leaves the classifier unchanged; moving a support vector moves the boundary.

Why the Constraint $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$?

The margin boundaries are $\mathbf{w}^\top \mathbf{x} + b = 1$ and $\mathbf{w}^\top \mathbf{x} + b = -1$. Therefore

$$y_i = +1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \geq 1, \quad y_i = -1 \Rightarrow \mathbf{w}^\top \mathbf{x}_i + b \leq -1.$$

Combining both cases into one expression:

$$y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1.$$

Support vectors

Points where this constraint holds with *equality*, $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$, are the **support vectors**: the closest points to the boundary, and the *only* points that determine it. Removing a non-support vector leaves the classifier unchanged; moving a support vector moves the boundary.

Limitations of the Maximal Margin Classifier

It requires perfect separability

Even a single misclassified observation makes a perfect separating hyperplane impossible.

- ▶ Real-world data often has noise, outliers, and overlapping classes.
- ▶ The classifier is highly sensitive to outliers, since the boundary depends only on the closest points.
- ▶ In many practical problems, no hyperplane perfectly separates all training samples.

The fix

Relax the strict separation requirement and allow a small number of margin violations — the **support vector classifier** (soft-margin classifier).

Limitations of the Maximal Margin Classifier

It requires perfect separability

Even a single misclassified observation makes a perfect separating hyperplane impossible.

- ▶ Real-world data often has noise, outliers, and overlapping classes.
- ▶ The classifier is highly sensitive to outliers, since the boundary depends only on the closest points.
- ▶ In many practical problems, no hyperplane perfectly separates all training samples.

The fix

Relax the strict separation requirement and allow a small number of margin violations — the **support vector classifier** (soft-margin classifier).

Limitations of the Maximal Margin Classifier

It requires perfect separability

Even a single misclassified observation makes a perfect separating hyperplane impossible.

- ▶ Real-world data often has noise, outliers, and overlapping classes.
- ▶ The classifier is highly sensitive to outliers, since the boundary depends only on the closest points.
- ▶ In many practical problems, no hyperplane perfectly separates all training samples.

The fix

Relax the strict separation requirement and allow a small number of margin violations — the **support vector classifier** (soft-margin classifier).

Support Vector Classifier: Soft Margins

Idea

Allow some observations to lie inside the margin, or even be misclassified, in exchange for a wider, more stable margin overall.

Slack variables

Introduce $\xi_i \geq 0$ for each observation i , and solve

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Support Vector Classifier: Soft Margins

Idea

Allow some observations to lie inside the margin, or even be misclassified, in exchange for a wider, more stable margin overall.

Slack variables

Introduce $\xi_i \geq 0$ for each observation i , and solve

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Understanding Slack Variables

ξ_i measures how much observation i violates its margin.

- ▶ $\xi_i = 0$: on or outside the margin, correctly classified.
- ▶ $0 < \xi_i < 1$: inside the margin, still correctly classified.
- ▶ $\xi_i = 1$: exactly on the decision boundary.
- ▶ $\xi_i > 1$: misclassified.

Larger ξ_i means a larger margin violation.

Understanding Slack Variables

ξ_i measures how much observation i violates its margin.

- ▶ $\xi_i = 0$: on or outside the margin, correctly classified.
- ▶ $0 < \xi_i < 1$: inside the margin, still correctly classified.
- ▶ $\xi_i = 1$: exactly on the decision boundary.
- ▶ $\xi_i > 1$: misclassified.

Larger ξ_i means a larger margin violation.

Understanding Slack Variables

ξ_i measures how much observation i violates its margin.

- ▶ $\xi_i = 0$: on or outside the margin, correctly classified.
- ▶ $0 < \xi_i < 1$: inside the margin, still correctly classified.
- ▶ $\xi_i = 1$: exactly on the decision boundary.
- ▶ $\xi_i > 1$: misclassified.

Larger ξ_i means a larger margin violation.

Understanding Slack Variables

ξ_i measures how much observation i violates its margin.

- ▶ $\xi_i = 0$: on or outside the margin, correctly classified.
- ▶ $0 < \xi_i < 1$: inside the margin, still correctly classified.
- ▶ $\xi_i = 1$: exactly on the decision boundary.
- ▶ $\xi_i > 1$: misclassified.

Larger ξ_i means a larger margin violation.

Understanding Slack Variables

ξ_i measures how much observation i violates its margin.

- ▶ $\xi_i = 0$: on or outside the margin, correctly classified.
- ▶ $0 < \xi_i < 1$: inside the margin, still correctly classified.
- ▶ $\xi_i = 1$: exactly on the decision boundary.
- ▶ $\xi_i > 1$: misclassified.

Larger ξ_i means a larger margin violation.

The Cost Parameter C

Trade-off

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

C trades off margin width against classification error — the **bias-variance tradeoff** in SVM form.

Large C

High penalty for violations; fewer misclassifications; narrower margin (low bias, high variance).

Small C

Low penalty for violations; more violations allowed; wider margin (high bias, low variance).

The Cost Parameter C

Trade-off

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

C trades off margin width against classification error — the **bias-variance tradeoff** in SVM form.

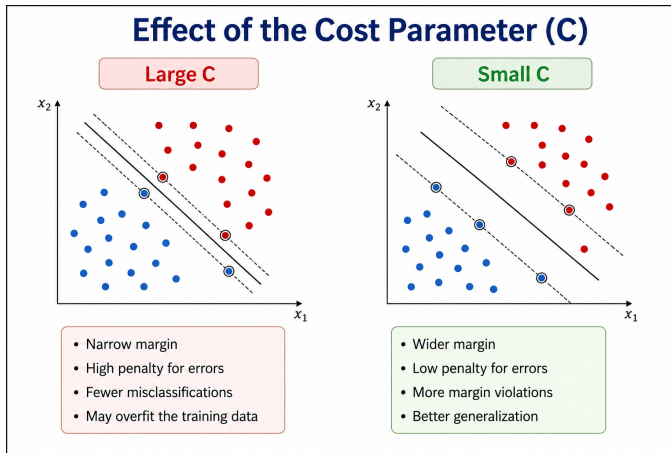
Large C

High penalty for violations; fewer misclassifications; narrower margin (low bias, high variance).

Small C

Low penalty for violations; more violations allowed; wider margin (high bias, low variance).

Effect of the Cost Parameter C



Why Do We Need Support Vector Machines?

The support vector classifier performs well for approximately *linear* decision boundaries, but many real-world datasets are not linearly separable.

- ▶ Complex datasets often have non-linear relationships.
- ▶ A straight hyperplane cannot separate such classes.
- ▶ Increasing the number of features may reveal a linear separation.
- ▶ SVMs achieve this using the **kernel trick**.

$$\mathbf{x} \longrightarrow \phi(\mathbf{x}) \longrightarrow \text{Linear Separation.}$$

Why Do We Need Support Vector Machines?

The support vector classifier performs well for approximately *linear* decision boundaries, but many real-world datasets are not linearly separable.

- ▶ Complex datasets often have non-linear relationships.
- ▶ A straight hyperplane cannot separate such classes.
- ▶ Increasing the number of features may reveal a linear separation.
- ▶ SVMs achieve this using the **kernel trick**.

$$\mathbf{x} \longrightarrow \phi(\mathbf{x}) \longrightarrow \text{Linear Separation.}$$

Why Do We Need Support Vector Machines?

The support vector classifier performs well for approximately *linear* decision boundaries, but many real-world datasets are not linearly separable.

- ▶ Complex datasets often have non-linear relationships.
- ▶ A straight hyperplane cannot separate such classes.
- ▶ Increasing the number of features may reveal a linear separation.
- ▶ SVMs achieve this using the **kernel trick**.

$$\mathbf{x} \longrightarrow \phi(\mathbf{x}) \longrightarrow \text{Linear Separation.}$$

Mapping to a Higher-Dimensional Feature Space

Given $\mathbf{x} = (x_1, x_2)$, a non-linear map $\phi : \mathbf{x} \rightarrow \phi(\mathbf{x})$ sends the data into a higher-dimensional feature space, e.g.

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2, x_2^2, x_1x_2).$$

- ▶ Data that is non-linearly separable in the original space may become linearly separable in ϕ -space.
- ▶ A linear hyperplane is fit there.
- ▶ Transformed back, it becomes a non-linear boundary in the original space.

Mapping to a Higher-Dimensional Feature Space

Given $\mathbf{x} = (x_1, x_2)$, a non-linear map $\phi : \mathbf{x} \rightarrow \phi(\mathbf{x})$ sends the data into a higher-dimensional feature space, e.g.

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2, x_2^2, x_1x_2).$$

- ▶ Data that is non-linearly separable in the original space may become linearly separable in ϕ -space.
- ▶ A linear hyperplane is fit there.
- ▶ Transformed back, it becomes a non-linear boundary in the original space.

The Kernel Trick

The problem

Explicitly computing $\phi(\mathbf{x})$ for every observation can be prohibitively expensive, especially in very high (or infinite) dimensions.

The trick

Use a **kernel function** that computes the inner product in the transformed space *directly*, without ever forming $\phi(\mathbf{x})$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j).$$

- ▶ Eliminates explicit computation of $\phi(\mathbf{x})$.
- ▶ Handles very high-dimensional feature spaces efficiently.
- ▶ Learns non-linear boundaries while all computation stays in the original feature space.

The Kernel Trick

The problem

Explicitly computing $\phi(\mathbf{x})$ for every observation can be prohibitively expensive, especially in very high (or infinite) dimensions.

The trick

Use a **kernel function** that computes the inner product in the transformed space *directly*, without ever forming $\phi(\mathbf{x})$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j).$$

- ▶ Eliminates explicit computation of $\phi(\mathbf{x})$.
- ▶ Handles very high-dimensional feature spaces efficiently.
- ▶ Learns non-linear boundaries while all computation stays in the original feature space.

The Kernel Trick

The problem

Explicitly computing $\phi(\mathbf{x})$ for every observation can be prohibitively expensive, especially in very high (or infinite) dimensions.

The trick

Use a **kernel function** that computes the inner product in the transformed space *directly*, without ever forming $\phi(\mathbf{x})$:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j).$$

- ▶ Eliminates explicit computation of $\phi(\mathbf{x})$.
- ▶ Handles very high-dimensional feature spaces efficiently.
- ▶ Learns non-linear boundaries while all computation stays in the original feature space.

Kernel	Function	Typical use
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$	Linearly separable data
Polynomial	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$	Polynomial boundary
RBF (Gaussian)	$K(\mathbf{x}, \mathbf{z}) = e^{-\gamma \ \mathbf{x} - \mathbf{z}\ ^2}$	Most common
Sigmoid	$K(\mathbf{x}, \mathbf{z}) = \tanh(\alpha \mathbf{x}^T \mathbf{z} + c)$	NN-inspired

Most widely used

The **RBF (Gaussian) kernel** is the default choice: it can model highly non-linear decision boundaries.

Kernel	Function	Typical use
Linear	$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$	Linearly separable data
Polynomial	$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$	Polynomial boundary
RBF (Gaussian)	$K(\mathbf{x}, \mathbf{z}) = e^{-\gamma \ \mathbf{x} - \mathbf{z}\ ^2}$	Most common
Sigmoid	$K(\mathbf{x}, \mathbf{z}) = \tanh(\alpha \mathbf{x}^T \mathbf{z} + c)$	NN-inspired

Most widely used

The **RBF (Gaussian) kernel** is the default choice: it can model highly non-linear decision boundaries.

Radial Basis Function (RBF) Kernel

Definition

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right),$$

where γ controls the influence of each training sample. The RBF kernel implicitly maps into an *infinite*-dimensional feature space.

Large γ

Complex decision boundary; may overfit.

Small γ

Smooth decision boundary; may underfit.

Radial Basis Function (RBF) Kernel

Definition

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right),$$

where γ controls the influence of each training sample. The RBF kernel implicitly maps into an *infinite*-dimensional feature space.

Large γ

Complex decision boundary; may overfit.

Small γ

Smooth decision boundary; may underfit.

Choosing the Parameter γ

Small γ	Large γ
Large influence region	Small influence region
Smooth decision boundary	Complex decision boundary
Higher bias	Lower bias
Lower variance	Higher variance
May underfit	May overfit

In practice

Both γ and C are chosen by cross-validation to achieve the best generalization performance.

Choosing the Parameter γ

Small γ	Large γ
Large influence region	Small influence region
Smooth decision boundary	Complex decision boundary
Higher bias	Lower bias
Lower variance	Higher variance
May underfit	May overfit

In practice

Both γ and C are chosen by cross-validation to achieve the best generalization performance.

SVMs With More Than Two Classes

SVMs are inherently **binary** classifiers. For $K > 2$ classes, two common strategies extend them:

One-vs-Rest (OvR)

Train one classifier per class, each distinguishing that class from all others. Total classifiers: K .

One-vs-One (OvO)

Train one classifier per pair of classes.
Total classifiers: $\frac{K(K-1)}{2}$.

In practice

Libraries such as `scikit-learn` default to the One-vs-One strategy for multiclass SVMs.

SVMs With More Than Two Classes

SVMs are inherently **binary** classifiers. For $K > 2$ classes, two common strategies extend them:

One-vs-Rest (OvR)

Train one classifier per class, each distinguishing that class from all others. Total classifiers: K .

One-vs-One (OvO)

Train one classifier per pair of classes.
Total classifiers: $\frac{K(K-1)}{2}$.

In practice

Libraries such as `scikit-learn` default to the One-vs-One strategy for multiclass SVMs.

SVMs With More Than Two Classes

SVMs are inherently **binary** classifiers. For $K > 2$ classes, two common strategies extend them:

One-vs-Rest (OvR)

Train one classifier per class, each distinguishing that class from all others. Total classifiers: K .

One-vs-One (OvO)

Train one classifier per pair of classes.
Total classifiers: $\frac{K(K-1)}{2}$.

In practice

Libraries such as `scikit-learn` default to the One-vs-One strategy for multiclass SVMs.

Advantages

- ▶ Effective in high-dimensional spaces.
- ▶ Works well with small training sets.
- ▶ Maximizes margin $\Rightarrow$ good generalization.
- ▶ Handles non-linear data via kernels.
- ▶ Memory efficient: uses only support vectors.

Limitations

- ▶ Choosing the right kernel can be difficult.
- ▶ Performance depends heavily on C and γ .
- ▶ Computationally expensive at large scale.
- ▶ Less interpretable than trees.
- ▶ Multiclass needs extra strategies (OvO/OvR).

In Practice: Fitting an SVM in a Few Lines

```
from sklearn.svm import SVC

# Linear SVC, cost parameter C
clf = SVC(kernel="linear", C=1.0)
clf.fit(X_train, y_train)

# Non-linear: RBF kernel, tune C & gamma
clf_rbf = SVC(kernel="rbf", C=1.0,
               gamma="scale")
clf_rbf.fit(X_train, y_train)

acc = clf_rbf.score(X_test, y_test)
```

What changes

Only the kernel argument distinguishes a linear support vector classifier from a fully non-linear SVM — the underlying margin-maximization problem is solved the same way in both cases.

Habit

Always tune C (and γ for RBF) by cross-validation; never take library defaults on faith.

Applications of Support Vector Machines

- ▶ **Text classification**: spam detection, sentiment analysis, document categorization.
- ▶ **Computer vision**: object detection, image classification.
- ▶ **Bioinformatics**: cancer diagnosis, gene/protein classification.
- ▶ **Finance & cybersecurity**: credit risk, fraud and intrusion detection.

SVM: a powerful supervised learner for both linear and non-linear classification.

- ▶ **Text classification**: spam detection, sentiment analysis, document categorization.
- ▶ **Computer vision**: object detection, image classification.
- ▶ **Bioinformatics**: cancer diagnosis, gene/protein classification.
- ▶ **Finance & cybersecurity**: credit risk, fraud and intrusion detection.

SVM: a powerful supervised learner for both linear and non-linear classification.



SECTION 7

Summary

Summary & What Comes Next

Key takeaways

- ▶ Maximal margin classifier: $\min \frac{1}{2} \|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$; only support vectors matter.
- ▶ Soft margin (support vector classifier): slack ξ_i and cost C trade margin width against violations.
- ▶ Kernel trick: $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ gives non-linear boundaries without ever forming $\phi(\mathbf{x})$.

Where this leads

Margin maximization is one way to learn a flexible decision boundary; another is to *learn the feature map itself* rather than fix a kernel by hand — the idea behind **neural networks**. As we move forward, keep this thread in mind: every method we've seen this course is a different answer to the same question — how to represent f , and how to fit it without overfitting.

Summary & What Comes Next

Key takeaways

- ▶ Maximal margin classifier: $\min \frac{1}{2} \|\mathbf{w}\|^2$ s.t. $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$; only support vectors matter.
- ▶ Soft margin (support vector classifier): slack ξ_i and cost C trade margin width against violations.
- ▶ Kernel trick: $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ gives non-linear boundaries without ever forming $\phi(\mathbf{x})$.

Where this leads

Margin maximization is one way to learn a flexible decision boundary; another is to *learn the feature map itself* rather than fix a kernel by hand — the idea behind **neural networks**. As we move forward, keep this thread in mind: every method we've seen this course is a different answer to the same question — how to represent f , and how to fit it without overfitting.

Support Vector Machines

From maximal margins to the kernel trick: geometry as a classifier

Questions & discussion welcome

